

DAY 2 • LEARNING JOURNAL



Explore Data and Extract Insights Using BigQuery

Day 2 Notes — Introduction to BigQuery, its Architecture, and How It Organizes Data

Data Warehouse

BigQuery Basics

Architecture

Slots & Pricing

Hands-on Lab

Compiled from personal study notes • Google Skills: "Introduction to Data Analytics on Google Cloud"

Quick Recap: Data Lake vs. Data Warehouse

Before jumping into BigQuery, it helps to reconnect with two ideas from Day 1, since BigQuery sits right in the middle of them.

Data Lake

A data lake is simply a large pool of **raw, unorganized, and unclassified data** that has no specific purpose yet. Think of it as a holding area where everything is dumped first and sorted out later.

Data Warehouse

A data warehouse contains **structured and organized data**, which can be used for advanced querying and reporting. As a data warehouse, BigQuery can store transformed data specifically for business insights.

 **How they connect:** Data sources feed into a data lake first, and are then processed into your data warehouse for analysis and reporting. So the journey usually looks like: raw data → data lake → cleaned and structured data → data warehouse (BigQuery) → insights.

 **Analogy:** A data lake is like a big box where you dump all your receipts, bills, and papers through the year without sorting them. A data warehouse is like a well-labelled filing cabinet where those same papers have been sorted into folders by month and category, ready for you to pull out exactly what you need.

BigQuery is specifically optimized for **running analytical queries over large datasets**. To put that in perspective:

Terabytes of data → queried in seconds

Petabytes of data → queried in minutes

 **Just to put that in perspective:** 1 terabyte is roughly equivalent to about 250,000 photos on your phone. 1 petabyte is 1,000 times bigger than that. Querying that much data in minutes, instead of hours or days, is what makes BigQuery so powerful for real businesses.

Introduction to BigQuery for Data Analytics

This module covers three learning goals:

1. Describe BigQuery and the BigQuery solution architecture.
2. Derive insights from data with BigQuery.
3. Describe how BigQuery organizes data.

What does it mean that BigQuery is a "fully managed" data warehouse?

It means BigQuery takes care of the underlying infrastructure for you, so you can focus purely on using SQL queries to answer business questions, without worrying about deployment, scalability, or security. In short: you bring the questions, BigQuery handles the plumbing.

 **Analogy:** Think of BigQuery like staying in a serviced apartment instead of owning a house. You don't need to worry about plumbing repairs, electricity wiring, or security systems — that's all managed for you. You just move in and live there. Similarly, with BigQuery, Google manages the servers, storage, and security, and you simply run your queries.

Federated queries

Federated queries let you **read data directly from external sources**, without first copying that data into BigQuery, while streaming support keeps that data continuously updated.

 **In simple words:** Instead of always bringing the data home, federated queries let BigQuery "visit" the data where it already lives and read it directly. This is the same idea as the data-source connector from Day 1.

BigQuery ML & BI Engine

BigQuery ML and BI Engine are two features that let you **analyze and understand** your data more deeply, directly inside BigQuery, without needing to export it to a separate machine learning tool.

Five Key Features of BigQuery

- 1 **Two services in one:** BigQuery can store and analyze petabytes of data, combining both a storage system and an analytics engine in a single product.
- 2 **Serverless:** BigQuery handles all the infrastructure behind the scenes, so you can focus entirely on running SQL queries instead of managing servers.
- 3 **Flexible pay-as-you-go pricing:** You pay only for what your query actually processes, or you can choose a flat-rate pricing option if you prefer predictable costs.
- 4 **Data encryption at rest by default:** Any data stored on disk is automatically encrypted, with no extra setup needed on your part.
- 5 **Built-in Machine Learning:** You can write ML models directly inside BigQuery using plain SQL, without needing to learn a separate ML programming framework.

 **What is "serverless" in plain terms?** It does not mean there are no servers at all — it means you never have to think about them. You don't set them up, patch them, or scale them; Google does that invisibly in the background while you simply write and run queries.

If you decide to use other professional tools, such as **Vertex AI** from Google Cloud, to train your ML model, you can export datasets from BigQuery directly into Vertex AI. This gives you a seamless integration across the entire journey from raw data to a working AI model.

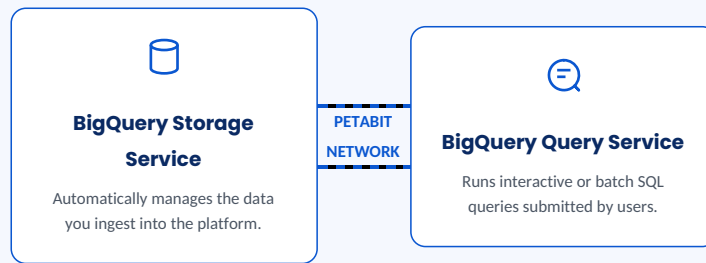
BigQuery Solution Architecture

What does a typical data warehouse solution architecture look like?

- BigQuery is the **analytics engine** that sits at the end of the data pipeline.
- It stores incoming data and then lets you perform analysis and model building on top of it.
- It is actually **two services in one**: a fast SQL query engine, and a fully managed storage layer for loading and storing your datasets.
- It maximizes flexibility by **separating the compute resources** that analyze your data from your storage choices.
- You can store and analyze data within BigQuery, or use BigQuery to query your data where it already lives (through federated queries).

How the two services are connected

Google's high-speed internal network connects these two services. This super-fast network is what enables BigQuery to separate compute and storage in the first place, while still working together seamlessly.



Conceptual diagram: Google's Petabit network connects the storage and query services internally.

What does each service do?

- The **storage service** automatically manages the data that you ingest into the platform. BigQuery organizes data tables into units called **datasets**, and these datasets are scoped to your Google Cloud project.
- When you reference a table from the command line, in SQL queries, or in code, you refer to it using the construct: **project.dataset.table**.
- Tables are stored as highly compressed columns in Google's **Colossus file system**, which provides durability and availability.
- The storage service supports both **bulk data ingest** and **streaming ingest**.
- The **query service** runs interactive or batch queries that are submitted through the Cloud Console, the BigQuery web UI, the bq command-line tool, or the REST API.
- The REST API is supported for several programming languages, so developers can integrate BigQuery into their own applications.

 **Analogy:** Think of a huge library building. The storage service is like the shelves and archive rooms that hold every book in an organized, compressed way. The query service is like the librarian who fetches exactly the books (data) you ask for, whether you ask in person, by phone, or through a written request form. Both work together, but they can each grow and improve independently.

Serverless resource allocation

- You do not need to provision resources before using BigQuery, unlike many traditional relational database management systems.
- It allocates storage and query resources **dynamically**, based on your actual usage patterns.
- Storage resources are allocated as you consume them, and deallocated as you remove data or drop tables.
- Query resources are allocated according to the query's type and complexity.

 **What is a "slot"?** A BigQuery slot is a virtual CPU used by BigQuery to execute SQL queries. BigQuery automatically calculates how many slots each query requires, depending on the size and complexity of that query. There is no minimum usage commitment, and by default, all BigQuery customers have access to 2,000 slots for query operations. You can also reserve a fixed number of slots for your project if you need guaranteed performance.

How BigQuery Organizes Data

BigQuery organizes data to make analytics easier. Everything follows a clear hierarchy, from the broadest level (project) down to the most detailed level (column):



Why structure information into datasets, projects, and tables?

The short answer: to structure your information logically. In more detail:

- You can use **multiple datasets** to separate tables that belong to different analytical domains, and use project-level scoping to isolate datasets from each other according to your business needs.
- You can align **projects to billing**, and use datasets for access control.
- You store data in separate tables based on **logical schema considerations**.

☰ **Analogy:** Think of a project like a company's office building, a dataset like a department within that building (say, Sales or HR), and a table like a specific filing cabinet inside that department. Just as you would not mix HR files with Sales files, BigQuery keeps data grouped logically so it stays organized and secure.

Table schemas

- Every table has a **schema**: a definition of what columns exist and what type of data each one holds.
- You can enter the schema manually through the Google Cloud console, or by supplying a JSON file.
- File formats like **Avro**, **Parquet**, **ORC**, Firestore export, or Datastore export are self-describing, so BigQuery automatically infers the table schema for you.

BigQuery as a "columnar store"

- BigQuery is called a **columnar store**, which means it is designed for processing columns, not rows.
- Since each column holds data of a single type, BigQuery can compress that column's data much more effectively than if rows were mixed together.
- Each column is automatically **compressed, encrypted, and replicated** to support the availability and durability of data in BigQuery.
- BigQuery stores data in a **massively distributed format**. The same underlying technology powers Google Cloud Storage and apps like Gmail and Ads.
- BigQuery auto-optimizes the balancing and partitioning of these data shards in the back end, so you rarely have to think about it yourself.

📊 **Row storage vs. column storage:** Imagine a spreadsheet of 1 million customers with columns for Name, Age, and City. A row-based system reads an entire row (Name, Age, City together) even if you only need the Age column. A columnar store like BigQuery reads just the Age column across all 1 million rows, which is much faster and compresses better, since all the values in that column are the same type.

Connecting to other services

- There are connectors to other services, such as **Dataproc**, which simplify creating complex workflows between BigQuery and other Google Cloud processing services.
- The query service can also run query jobs on data contained in other locations, such as tables in CSV files hosted in Cloud Storage, or data in Google Sheets.
- The storage service and the query service work together internally to organize data and make queries efficient, even over huge datasets that are petabytes in size.

☀️ **Best practice:** One of the most important controls you have over resource consumption and cost is controlling the amount of data processed. When exploring a new dataset, start broad, and then narrow your focus to the specific fields and rows that you actually need. This keeps both your queries fast and your bill low.

The BigQuery User Interface

- 1 You open BigQuery using the **Navigation menu** in the Google Cloud console.
- 2 Every resource in Google Cloud exists inside a **project**, which you can view directly from the console.
- 3 In addition to security and access control, a project is what links the usage of a resource to a **credit card**; it is what makes a resource billable.

- 4 BigQuery has numerous **public datasets** that anyone can query, free of charge, to practise and explore.
- 5 One well-known example is a dataset containing **all public Wikipedia page metadata**.

Below is a simplified, conceptual illustration of what the BigQuery console interface generally looks like, just to visualize the idea (not an exact screenshot):

•••

▪ Explorer

my-project

bigquery-public-data

wikipedia

▪ Saved queries

▪ Query history

```
SELECT title, views
FROM `wikipedia.pageviews`
ORDER BY views DESC
LIMIT 5;
```

title	views
Main_Page	3,204,112
Special:Search	1,987,304
India	842,110

Conceptual mockup of a BigQuery query against a public dataset — illustrative only.

Hands-on Lab: BigQuery Qwik Start (Console)

This lab is a first hands-on walkthrough of the BigQuery console. In this lab, the following four tasks are covered:

✓ Step 1 & 2

Query a **public dataset** to get comfortable with the query editor, then **create a new dataset** of your own to hold custom tables.

✓ Step 3 & 4

Load data into a new table inside that dataset, and then **query the custom table** you just created.

 **Why this matters:**

This simple four-step flow (query → create → load → query again) is the same basic loop you will repeat in almost every real BigQuery project: explore existing data, set up your own space, bring your data in, and then start asking questions of it.

Notes compiled from handwritten study notes • Introduction to Data Analytics on Google Cloud • Day 2